

Planning Stage Requirements

Contents

Background	1
Assignment.....	1
Game Direction and Target Audience.....	2
User Interviews	2
Requirements Analysis.....	3
Mock-Ups	4
Team Presentation and Pitch	4
Submission	5
Grading.....	5

Background

During the planning stage of the Agile Software Development Life Cycle, a project's concept is expanded on, requirements are solidified, and a team is formed that will prepare for development. It is a major liability to jump right into developing software without doing comprehensive planning and market research. Otherwise, there are dangers of implementing incorrect/inaccurate requirements and ending up with less useful software, or even worse – software that doesn't reach its target audience correctly and fails to provide value.

Before each team can begin to work on developing their game, there are several pieces that need to be planned out first to ensure development starts in the right spot. While I know many teams are eager to just jump right in, there is really nothing to get started on yet when the game's overall direction and target audience hasn't even been decided on yet. How can any work be getting done before the work to get done has been decided on?

In our workplace simulation, your team is looking to get funded in order to develop a game. **The goal of this “planning stage” is to come up with a solidified plan of attack for creating a game, alongside sufficient backing research and data to convince me to approve and give your team funding.**

This planning stage will start on 9/1, and end on 9/10 with the team's presentation and pitch. No code should be written during the planning stage.

Assignment

The below sections all need to be completed during a team's planning stage. They are intended to be carried out in order.

Game Direction and Target Audience

Each team must choose the overall direction they want to develop the game in. They must also figure out the (hypothetical) target audience they are developing the game for. The game can be taken in any direction as long as there is a userbase to match it.

Your game direction should be as clear as possible, but specific implementation details do not need to be decided on at this time – keep things at a higher level. The overall “vision” is important in order to structure the iterative development stages and ensure the correct goal is being worked towards.

Target audience is VERY important. If the game doesn’t have an intended userbase that it is explicitly being made for, it will likely fall flat. Some video game companies try to make their games fit many different userbases (most Nintendo games match this), while some try to appeal to a select userbase (such as the Dark Souls series). Some even try to combine multiple userbases that otherwise have nothing in common (Fortnite has 8-year-old whiney kids playing against adults that make a living through playing the game).

For example, if you wanted to turn the platformer game into a more challenging game that included more enemies, level obstacles, etc., your future users could simply be people that like challenging classic platformer video games (or to get even more specific, people that are fans of other specific challenging platformer games such as Celeste or Super Meat Boy). As another example, you could create a game based off an entirely original idea, and your target audience could be gamers looking for something new within a specific genre. Do not worry about novelty, you don’t need to come up with something that has “never been done before” – there is nothing wrong with wanting to make a game inspired by something else, or doing your own “take” on an already established genre.

To help with ideation/brainstorming, you can see the projects that previous SER225 teams have created on my website [here](#).

User Interviews

While we all think that we *know* what other people want, it is important to conduct actual user testing and hear directly from your team’s userbase. What do they think of the game idea your team has come up with? Does it meet their wants/needs? This first-hand information will allow your team to either confirm the chosen direction, make tweaks to it to conform to the audience’s wants/needs, or possibly even lead to major changes if it completely misses the mark. Otherwise, there is risk that your team will be developing software that does not meet the needs of your target audience.

Each team member needs to conduct one user interview. This user must match your target audience demographic. Ideally, the users chosen should represent the “ranges” that your target audience includes. For example, if your team is creating a family friendly game intended for all ages, then several users of different age ranges should be chosen.

Please refer to the **User Interview Instructions.pdf** document to see the eligibility rules for an interviewee, as well as how to properly conduct/format a user interview.

Each team member should then summarize the results of their user interview and present the information to the rest of the team to see if anything that came from it is worth analyzing/investigating. It’s possible that a team’s vision/end goal gets added on to, tweaked, or even considerably changed as a result of these interviews.

Requirements Analysis

Requirements are descriptions of required features and functionalities for the product being delivered.

The team should decide on key requirements that the game must have for it to match the direction and meet the end goal. Typically, “Day 1 Requirements” are requirements decided upon from the get-go. The more requirements that can be decided on at this stage, the better. However, that doesn’t mean that these requirements cannot be changed or even omitted in the future as development takes place.

For example, if I am starting with the platformer game and my end goal is to create a much longer, more creative, and more challenging experience, some requirements I might have based on how I want the game to function could be “create an enemy that explodes if it gets near the player”, “create a level that involves forcing the player to make very tough jumps”, “make it so the player can jump off walls”, “setup a system to allow for changing to a new level after a level is cleared”, etc.

Notice how the above example requirements do not mention specific implementation details; there is no mention to how these features actually get coded (if statements, classes, etc.) or how the task is to actually be carried out. Generally, requirements come from the customers that want the product or are derived from customers’ wants/needs (based on canvassing/user interviews) – these requirements represent their POV. Customers likely do not explicitly care HOW a requirement is implemented; they just care that the product meets the requirement.

Something else to keep in mind is that these Day 1 requirements should be “bigger picture” items. Little things or “nitty gritty details” don’t really need to be included up front. For example, requirements like “make player jump higher” or “make player run faster” are a bit too small and specific to need to be defined here. Sometimes it’s better to keep things vaguer if there is no clear-cut decision made yet. For example, a team could have the requirement “include a collectable of some kind that the player can collect throughout each level”, which is vaguer than something like “allow the player to collect coins throughout each level”. Both are fine and valid, but if the team hasn’t yet decided on the details of the collectable, keeping it vague on Day 1 and making the actual decision later on is okay.

I recommend teams take great care into writing out their requirements, as the clearer these are, the easier time you all will have breaking them down and implementing them later on. Since you will be developing this software over the course of the semester, planning a sufficient number of requirements up front is expected. Furthermore, having more requirements earns trust from the stakeholder(s), as it shows the team has already put a lot of thought and effort into the plan they intend to carry out.

Note that a team having a lot of requirements does not mean that a team will be doing “more work” compared to other teams this semester. My standard for amount of work that gets done is the same across all teams and projects. If a team has requirements that are challenging for their level of experience or require a lot of development time, that will be factored into how much work I expect them to get done during each of the future iteration cycles. For example, if a team were to have a very difficult and time intensive requirement such as “make game have online multiplayer”, that is something I would expect to be broken down into small pieces and worked on over a longer period of time. I encourage each team to make this game the way they want without worry about future workload. It may actually hurt a team to try to intentionally keep things super “easy”, as I will then start throwing requirements at you to fill your time, and who knows what kind of insane requirements I’ll come up with.

Mock-Ups

A mock-up is a model or replica that is created to represent an entity for some purpose. For software development, it's common to create mock-ups to show off the desired end goal, or to allow visualization of how a requirement will look once it is implemented.

For our classroom workplace simulation, this is going to look a lot different than it does in actual industry. Normally this step is done by trained designers, and it takes a great deal of time and effort to do this. It's not as easy as it sounds – mock-ups can be really complicated and comprehensive, especially in cases of large software systems. It's also not something a software engineer is typically involved with creating, outside of giving advice based on the technical implications of the design. Software engineers often USE mock-ups as reference points to ensure they implement requirements correctly, but it is not typically their job to create them.

So instead, just to acknowledge the importance of this stage while also keeping in mind what you all are here for/have time for, your team is only required to create three simple “mock-ups” representing your end goal or specific requirements. These “mock-ups” can come in the form of pictures (such as Microsoft Paint masterpieces), or images from other media that matches a requirement. You could even show a GIF/Video from another video game or application – for example, if your team has the requirement “enemies can be killed when jumped on”, showing a clip of Mario jumping on an enemy and killing it would work well. These do not need to be perfect and you do not need to encompass everything in these three mock-ups, just please put thought and effort into them to show that this important step in the planning process is being carried out and taken seriously.

Tip: [this](#) website is very good for finding graphics from existing video games, which can potentially aid you in creating mock ups.

Team Presentation and Pitch

During class on 9/10, each team will present their pitch to me and the rest of the class. The presentation should include information, insights, and processes that the team used from all of the above planning stage steps in order to reach their desired end goal/vision. This includes going over their chosen game direction and target audience, user interviews and what was learned from them, requirements analysis process, mock-ups, and finalized “pitch” for what your team is planning on developing this semester.

The goal of this pitch is to convince me to approve and give your team funding in our class workplace simulation. In our workplace simulation, I will only offer your team funding if I feel confident in your team's vision and plans, as well as trust that your team has what it takes to carry out the plan and create the game.

After your team's pitch presentation, I will either approve funding and allow your team to advance to the first development cycle, or deny and you will fail the planning stage. Ideally, I will give a team the “informal” green light before the presentation takes place through routine check-ins during classes leading up to the pitch day. If a team waits until the last minute to do everything, they are putting themselves at risk to not be able to get funding and move on from the planning stage...yikes.

A team's presentation and pitch can be structured however they'd like as long as it includes all the required information. The key is to focus on the most important and relevant pieces of info, and make it easy for your audience to digest –inform, don't overwhelm! Any medium can be used, such as

PowerPoint. The presentation must not exceed ten minutes in length. Each member of a team must have a part in the presentation. After the presentation, the floor will be open to questions from the class. I will likely ask a question or two.

Also, you must come up with a team name! Your team's name cannot be the same name as another team from your section, another section, or any previous years. There are also some team names I do not allow to be chosen. You can check the **Team_Names_Reference.pdf** document to verify that your desired team name is acceptable.

It is required that all team members attend class on 9/10 for team pitch presentations. Each team member must participate and speak in their team's presentation. Since this is the first presentation for the class, and not everyone fully knows each other yet, it is a good idea to quickly introduce the team's name and each member of the team at the beginning of the presentation.

Submission

The following must be submitted to Blackboard prior to a team's class section time on 9/11:

- Document highlighting game direction and target audience decision
- User interview Zoom recordings (should be one per team member)
- User interview summaries/learnings (can be one per team member or done as a collective)
- Day 1 Requirements
- Three Mock-Ups
- Any presentation materials (such as PowerPoints).

Only one team member should submit everything for the team to Blackboard. **Please do not submit items across multiple team members.**

Grading

The Planning Stage is worth 10% of a team's overall course grade. A grade will be given out to the team after they have finished the planning stage.

For a team to receive an A, all steps of the planning stage must have been carried out correctly, and their pitch must be approved. Following directions and taking care in each step of the process is the key. I am not expecting perfection, but I am expecting a high level of effort and attention to detail. If any of the required steps are not submitted by the due date, the highest grade a team can earn is a C. Late work will not be accepted unless explicitly excused by myself. If a team does not present on 9/10, they will receive an F. If a specific team member does not present on 9/10 and did not have an excused absence, that specific team member will receive an F.

I highly recommend that before a team's presentation on 9/10, they check-in with me at least once to ensure they are on the right track. If a team is not on track to receive an A, I will tell you exactly what you need to do. If a team is on track to receive an A, I will tell you that you are doing exactly what you need to do. If a team waits until the last minute to start working on this, there will likely not be time for me to check-in with them to ensure they are on the right track.